ONLY USE T4 GPU

```
!pip install transformers datasets stanza openpyxl nltk pandas tqdm
```

```
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025
Collecting emoji (from stanza)
  Downloading emoji-2.15.0-py3-none-any.whl.metadata (5.7 kB)
Requirement already satisfied: protobuf>=3.15.0 in /usr/local/lib/python3.12/dist-packages (from stanza) (5.29.5)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from stanza) (3.6)
Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.12/dist-packages (from stanza) (2.9.0+cu126)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-packages (from openpyxl) (2.0.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformer
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (202
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza) (1.14.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza) (3.1.6)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0-
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0-
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->sta
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->sta
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->star
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->st
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->st
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->star
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->sta
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->stanza) (3.5.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspe
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.13.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.13.0->stanza)
Downloading stanza-1.11.0-py3-none-any.whl (1.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/1.7 MB 33.3 MB/s eta 0:00:00
Downloading emoji-2.15.0-py3-none-any.whl (608 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 608.4/608.4 kB 19.3 MB/s eta 0:00:00
Installing collected packages: emoji, stanza
Successfully installed emoji-2.15.0 stanza-1.11.0
```

```
import pandas as pd
from datasets import load_dataset
from pathlib import Path
import IPython.display as display
from collections import Counter
import re
from nltk.corpus import stopwords
from nltk.stem.isri import ISRIStemmer
import nltk
import stanza
from tqdm import tqdm
import os
import numpy as np
from transformers import AutoTokenizer, AutoModel, AutoModelForMaskedLM, AutoModelForCausalLM
import torch
import math
```

```
# Creating a folder for processed data so the code doesn't crash later
os.makedirs("data/processed", exist_ok=True)
os.makedirs("data/raw", exist_ok=True)

print("ready")
```

```
ready
```

```
# تحميل الداتا من Hugging Face
dataset = load_dataset("KFUPM-JRCAI/arabic-generated-abstracts")

print("keys:")
print(dataset.keys())

# تقسيم الداتا
by_polishing = dataset["by_polishing"]
from_title = dataset["from_title"]
from_title_and_content = dataset["from_title_and_content"]

# تحويل لبيانات بايثون
by_polishing_df = by_polishing.to_pandas()
from_title_df = from_title.to_pandas()
from_title_and_content_df = from_title_and_content.to_pandas()
dataset_df = pd.concat([by_polishing_df, from_title_df, from_title_and_content_df], ignore_index=True)

# Save CSVs (As per original code logic)
by_polishing_df.to_csv("by_polishing.csv", index=False, encoding="utf-8-sig")
from_title_df.to_csv("from_title.csv", index=False, encoding="utf-8-sig")
from_title_and_content_df.to_csv("from_title_and_content.csv", index=False, encoding="utf-8-sig")
dataset_df.to_csv("dataset.csv", index=False, encoding="utf-8-sig")
print("تم حفظ البيانات في CSV")

# Re-read CSVs (Original Logic)
by_polishing = pd.read_csv('by_polishing.csv', encoding="utf-8-sig")
from_title = pd.read_csv('from_title.csv', encoding="utf-8-sig")
from_title_and_content = pd.read_csv('from_title_and_content.csv', encoding="utf-8-sig")
dataset = pd.read_csv('dataset.csv', encoding="utf-8-sig")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:  7.69k/? [00:00<00:00, 617kB/s]
data/by_polishing-00000-of-00001.parquet: 100%  8.49M/8.49M [00:00<00:00, 10.9MB/s]
data/from_title-00000-of-00001.parquet: 100%  6.90M/6.90M [00:00<00:00, 32.2MB/s]
data/from_title_and_content-00000-of-000(…): 100%  7.01M/7.01M [00:00<00:00, 36.1MB/s]
Generating by_polishing split: 100%  2851/2851 [00:00<00:00, 4783.19 examples/s]
Generating from_title split: 100%  2963/2963 [00:00<00:00, 8550.29 examples/s]
Generating from_title_and_content split: 100%  2574/2574 [00:00<00:00, 8498.91 examples/s]
keys:
dict_keys(['by_polishing', 'from_title', 'from_title_and_content'])
تم حفظ البيانات في CSV
```

```
# إنشاء داتا مصنفة (Human vs AI)

#by polishing method
human_df_by_polishing = pd.DataFrame({
    'text': by_polishing['original_abstract'],
    'label': 'human',
    'genration_method':'by_polishing'
})

ai_df_by_polishing = pd.DataFrame({
    'text': pd.concat([
        by_polishing['allam_generated_abstract'],
        by_polishing['jais_generated_abstract'],
        by_polishing['llama_generated_abstract'],
```

```
        by_polishing['openai_generated_abstract']
    ]),
    'label': 'ai',
    'genration_method':'by_polishing'
})

#from title method
human_df_from_title = pd.DataFrame({
    'text': from_title['original_abstract'],
    'label': 'human',
    'genration_method':'from_title'
})

ai_df_from_title = pd.DataFrame({
    'text': pd.concat([
        from_title['allam_generated_abstract'],
        from_title['jais_generated_abstract'],
        from_title['llama_generated_abstract'],
        from_title['openai_generated_abstract']
    ]),
    'label': 'ai',
     'genration_method':'from_title'
})

#from_title_and_content method
human_df_from_title_and_content = pd.DataFrame({
    'text': from_title_and_content['original_abstract'],
    'label': 'human',
    'genration_method':'from_title_and_content'
})

ai_df_from_title_and_content = pd.DataFrame({
    'text': pd.concat([
        from_title_and_content['allam_generated_abstract'],
        from_title_and_content['jais_generated_abstract'],
        from_title_and_content['llama_generated_abstract'],
        from_title_and_content['openai_generated_abstract']
    ]),
    'label': 'ai',
    'genration_method':'from_title_and_content'
})

# Dataframe merge
merged_df_dataset = pd.concat([human_df_by_polishing, ai_df_by_polishing,human_df_from_title, ai_df_from_title,human_df_from_ti
merged_df_by_polishing = pd.concat([human_df_by_polishing, ai_df_by_polishing], ignore_index=True)
merged_df_from_title = pd.concat([human_df_from_title, ai_df_from_title], ignore_index=True)
merged_df_from_title_and_content = pd.concat([human_df_from_title_and_content, ai_df_from_title_and_content], ignore_index=True

# Remove duplicates
merged_df_dataset = merged_df_dataset.drop_duplicates(ignore_index=True)

# Save to Excel (Original Logic)
with pd.ExcelWriter("data/raw/all_datasets.xlsx", engine="openpyxl") as writer:
    merged_df_by_polishing.to_excel(writer, sheet_name="by_polishing", index=False)
    merged_df_from_title.to_excel(writer, sheet_name="from_title", index=False)
    merged_df_from_title_and_content.to_excel(writer, sheet_name="from_title_and_content", index=False)
    merged_df_dataset.to_excel(writer, sheet_name="Merged_Full_Dataset", index=False)

print("Initial Merge Complete and Saved.")
```

```
Initial Merge Complete and Saved.
```

CHANGE 1

```
# ==========================================
# PHASE 1: STRUCTURAL FEATURES & PREPROCESSING
# ==========================================

# Reloading the dataframe from the previous step (just to be safe)
if 'merged_df_dataset' in locals():
    df = merged_df_dataset.copy()
elif 'df' not in locals():
    # Fallback if variable was lost
    df = pd.read_excel("data/raw/all_datasets.xlsx", sheet_name="Merged_Full_Dataset")
```

```
print(f"Working with {len(df)} rows.")

# ----------------------------------------------------------
# STEP 1: CALCULATE STRUCTURAL FEATURES (ON RAW TEXT)
# ----------------------------------------------------------
print("1. Calculating Structural Features (Paragraphs, Sentences, Diacritics)...")

def get_structural_features(text):
    if not isinstance(text, str) or not text.strip():
        # Added 0 at the end for num_paragraphs
        return pd.Series([0, 0, 0, 0, 0, 0])

    # 1. Diacritic Ratio (Feature 101)
    diacritics = re.findall(r'[\u064B-\u0652]', text)
    diacritic_ratio = (len(diacritics) / len(text)) * 100

    # 2. Paragraphs (Feature 35/38)
    paragraphs = [p.strip() for p in re.split(r'[\r\n]+', text) if p.strip()]
    num_paragraphs = len(paragraphs) if paragraphs else 1
    chars_per_para = len(text.replace(" ", "")) / num_paragraphs

    # 3. Sentences (Feature 34/39/80)
    sentences = re.split(r'[.!؟\n]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]
    num_sentences = len(sentences) if sentences else 1

    words = text.split()
    num_words = len(words)

    # Avg Words per Sentence (Feature 39)
    avg_words_per_sent = num_words / num_sentences

    # Flesch-Kincaid (Feature 80)
    syllables = sum(len(re.findall(r'[aeiouAEIOU\u0627\u0648\u064A]', w)) for w in words)
    if num_words > 0:
        flesch_score = 0.39 * (num_words / num_sentences) + 11.8 * (syllables / num_words) - 15.59
    else:
        flesch_score = 0

    # ADDED: num_paragraphs to return
    return pd.Series([diacritic_ratio, chars_per_para, avg_words_per_sent, flesch_score, num_sentences, num_paragraphs])

# Apply structurally - ADDED 'feat_35_num_paragraphs' to the list
df[['feat_101_diacritic_ratio', 'feat_38_chars_per_para',
    'feat_39_avg_words_per_sentence', 'feat_80_flesch_kincaid',
    'feat_34_num_sentences', 'feat_35_num_paragraphs']] = df['text'].apply(get_structural_features)


# ----------------------------------------------------------
# STEP 2: CLEAN TEXT (DESTRUCTIVE STEP)
# ----------------------------------------------------------
print("2. Cleaning Text (Removing Punctuation & Diacritics)...")

nltk.download('stopwords', quiet=True)
from nltk.stem.isri import ISRIStemmer
stemmer = ISRIStemmer()
arabic_stopwords = set(stopwords.words('arabic'))

def preprocess_text_v2(text):
    if not isinstance(text, str): return ""

    # Normalize Alef
    text = re.sub(r'[إأآ]', 'ا', text)
    text = re.sub(r'ى', 'ا', text)
    text = re.sub(r'ـ', '', text) # Tatweel

    # Remove non-Arabic letters (This removes punctuation!)
    #text = re.sub(r'[^\u0621-\u064A\s]', '', text)

    # Fixed to include letters, spaces, Western digits (0-9), and Eastern digits (٠-٩)
    text = re.sub(r'[^\u0621-\u064A\u0660-\u0669\s0-9]', '', text)

    # Stopwords & Stemming
    words = text.split()
    words = [stemmer.stem(w) for w in words if w not in arabic_stopwords]

    return ' '.join(words)
```

```
df["processed_text"] = df["text"].apply(preprocess_text_v2)


# ------------------------------------------------------------
# STEP 3: CALCULATE WORD-BASED FEATURES (ON CLEAN TEXT)
# ------------------------------------------------------------
print("3. Calculating Word-Based Features (Yule's K, etc.)...")

# Feature 17: Yule's K
def yules_k_measure(text):
    words = text.split()
    N = len(words)
    if N == 0: return 0
    freqs = Counter(words)
    numerator = sum(f*(f-1) for f in freqs.values())
    return 10000 * numerator / (N*N)

df["feat_17_yules_k"] = df["processed_text"].apply(yules_k_measure)

# Feature 18: Simpson's Diversity
def simpsons_diversity(text):
    words = text.split()
    N = len(words)
    if N == 0: return 0
    freqs = Counter(words)
    numerator = sum(f*(f-1) for f in freqs.values())
    denominator = N * (N - 1)
    return 1 - (numerator / denominator) if denominator > 0 else 0

df["feat_18_simpsons"] = df["processed_text"].apply(simpsons_diversity)

print("\n✓ Phase 1 Complete.")
display.display(df[['text', 'processed_text', 'feat_35_num_paragraphs', 'feat_17_yules_k']].head(3))
```

```
Working with 41937 rows.
1. Calculating Structural Features (Paragraphs, Sentences, Diacritics)...
2. Cleaning Text (Removing Punctuation & Diacritics)...
3. Calculating Word-Based Features (Yule's K, etc.)...

✓ Phase 1 Complete.
```

	text	processed_text	feat_35_num_paragraphs	feat_17_yules_k
0	كثيرا ما ارتبطت المصادر التاريخية في الأندلس خ...	ربط صدر ارخ دلس كتب رجم هرس رمج وغر درس حية عل...	1.0	115.436849
1	يعد العامل الثقافي احد ابرز الاسباب التي يعزى ...	يعد عمل ثقف احد برز سبب يعز سقط دول وحد حتا ان...	1.0	51.775148
2	شكلت تلك الجهود والمساعي الرائدة التي قام بها ...	شكل جهد سعي رند قدة ثور خلل رحل اول 19541956 ب...	1.0	84.199272

```
# ========================================
# FINAL REPORT: READ FROM RAW XLSX
# ========================================
import pandas as pd
import re
from collections import Counter

# 1. Read directly from the raw xlsx file
file_path = "/content/Final_Engineered_Dataset.xlsx"
print(f"\nReading data directly from: {file_path} ...")
df_final = pd.read_excel(file_path)

print("\n" + "="*40)
print("     (من الملف الخام) ملخص جودة البيانات")
print("="*40)

# 2. Missing Values (عدد القيم المفقودة)
print("\n:عدد القيم المفقودة")
cols_to_check = ['text', 'label', 'genration_method']
# Filter columns that actually exist in the file
present_cols = [c for c in cols_to_check if c in df_final.columns]
print(df_final[present_cols].isnull().sum())

# 3. Duplicates (عدد الصفوف المكررة)
print("\n:عدد الصفوف المكررة")
print(df_final.duplicated().sum())

# 4. Unique Labels (القيم الفريدة في label)
```

```
print("\n:label القيم الفريدة في")
if 'label' in df_final.columns:
    print(df_final['label'].unique())

# 5. Strange Symbols Calculation (using 'text' column)
def get_strange_chars(text):
    # Ensure text is string; Regex finds Non-Arabic, Non-Number, Non-Space
    if not isinstance(text, str): return []
    return re.findall(r"[^0-9\u0600-\u06FF\s]", text)

# Flatten list of all strange characters found
all_strange_tokens = [char for text in df_final['text'] for char in get_strange_chars(text)]
strange_counts = Counter(all_strange_tokens)
unique_symbols = sorted(strange_counts.keys())
total_strange_count = sum(strange_counts.values())

# 6. Strange Symbols Output (الرموز الغريبة الموجودة)
print("\n:(بدون تكرار) الرموز الغريبة الموجودة")
# Join them with commas and wrap in braces to match screenshot style
print(f"{{{', '.join([repr(s) for s in unique_symbols])}}}")

# 7. Total Strange Symbols (مجموع الرموز الغريبة)
print("\n:مجموع الرموز الغريبة")
print(total_strange_count)
print("="*40)
```

```
Reading data directly from: /content/Final_Engineered_Dataset.xlsx ...

========================================
     ملخص جودة البيانات (من الملف الخام)
========================================

:عدد القيم المفقودة
text                0
label               0
genration_method    0
dtype: int64

:عدد الصفوف المكررة
0

:label القيم الفريدة في
['human' 'ai']

:الرموز الغريبة الموجودة (بدون تكرار)
{'!', '"', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', 'A', 'B', 'C', 'D', 'E', 'F

:مجموع الرموز الغريبة
268918
========================================
```

CHANGE 2

```
# ==========================================
# PHASE 2: Deep Learning Models and Stanza CORRECTED
# ==========================================

import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForMaskedLM, AutoModelForCausalLM
import gc
from tqdm import tqdm
import math
import os
import pandas as pd

# OPTIONAL: Helps with fragmentation
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

# Setup Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Running on device: {device}")

# ---------------------------------------------------------
# PART A: AraBERT Features (Feat 59, 60 & 98)
# ---------------------------------------------------------
print("\n--- Loading AraBERT (for Features 59, 60 & 98) ---")
```

```
bert_model_name = "aubmindlab/bert-base-arabertv02"

try:
    bert_tokenizer = AutoTokenizer.from_pretrained(bert_model_name)
    # LOAD IN HALF PRECISION (float16) to save memory while keeping high batch size
    bert_model = AutoModel.from_pretrained(bert_model_name, dtype=torch.float16).to(device)
    bert_model.eval()
    print("✓ AraBERT Loaded (Float16).")
except Exception as e:
    print(f"Error loading AraBERT: {e}")

# Feat 59 & 60: Unique Embedding Words (500 and 1000)
# BATCH: 1024 (Kept as requested)
def get_embedding_uniqueness_both(text_list, batch_size=1024):
    results_500 = []
    results_1000 = []
    total = len(text_list)
    for i in tqdm(range(0, total, batch_size), desc="Feature 59 & 60 (Embeddings)"):
        batch_texts = text_list[i:i+batch_size]
        # We process 1000 tokens max here to cover both features
        enc = bert_tokenizer(batch_texts, return_tensors="pt", padding=True, truncation=True, max_length=1000).to(device)
        input_ids = enc["input_ids"]
        for j in range(len(batch_texts)):
            # Feature 59 (First 500)
            tokens_500 = input_ids[j][:500]
            decoded_500 = bert_tokenizer.decode(tokens_500, skip_special_tokens=True)
            results_500.append(len(set(decoded_500.split())))

            # Feature 60 (First 1000)
            tokens_1000 = input_ids[j][:1000]
            decoded_1000 = bert_tokenizer.decode(tokens_1000, skip_special_tokens=True)
            results_1000.append(len(set(decoded_1000.split())))

    return results_500, results_1000

# Feat 98: CLS Score
# BATCH: 512 (Kept as requested)
def get_cls_score(text_list, batch_size=512):
    results = []
    total = len(text_list)
    # Using autocast for safety, though model is already float16
    with torch.no_grad(), torch.amp.autocast('cuda'):
        for i in tqdm(range(0, total, batch_size), desc="Feature 98 (CLS Score)"):
            batch_texts = text_list[i:i+batch_size]
            enc = bert_tokenizer(batch_texts, return_tensors="pt", padding=True, truncation=True, max_length=512).to(device)
            outputs = bert_model(**enc)
            cls_embeddings = outputs.last_hidden_state[:, 0, :]
            scores = torch.mean(torch.abs(cls_embeddings), dim=1).cpu().numpy().tolist()
            results.extend(scores)
    return results

# --- FIX: Use RAW TEXT for Deep Learning ---
print("Using RAW text for Deep Learning features...")
texts = df['text'].astype(str).tolist()

# Calculate Feature 59 and 60
feat59, feat60 = get_embedding_uniqueness_both(texts, batch_size=1024)
df['feat_59_unique_embedding_words_500'] = feat59
df['feat_60_unique_embedding_words_1000'] = feat60

# Calculate Feature 98
df['feat_98_bert_cls_score'] = get_cls_score(texts, batch_size=512)

# FREE MEMORY
del bert_model
del bert_tokenizer
torch.cuda.empty_cache()
gc.collect()
print("✓ AraBERT features done & memory cleared.")


# ------------------------------------------------------------
# PART B: AraGPT2 Features (Feat 81 - LogRank)
# ------------------------------------------------------------
print("\n--- Loading AraGPT2 (for Feature 81 LogRank) ---")
gpt_model_name = "aubmindlab/aragpt2-medium"
```

```
try:
    gpt_tokenizer = AutoTokenizer.from_pretrained(gpt_model_name)
    # LOAD IN HALF PRECISION (float16)
    gpt_model = AutoModelForCausalLM.from_pretrained(gpt_model_name, dtype=torch.float16).to(device)
    gpt_model.eval()
    print("✓ AraGPT2 Loaded (Float16).")
except Exception as e:
    print(f"Error loading AraGPT2: {e}")

# Feat 81: LogRank
def calculate_logrank(text_list, batch_size=8):
    results = []
    gpt_tokenizer.pad_token = gpt_tokenizer.eos_token

    with torch.no_grad(), torch.amp.autocast('cuda'):
        for i in tqdm(range(0, len(text_list), batch_size), desc="Feature 81 (LogRank)"):
            batch_texts = text_list[i:i+batch_size]
            inputs = gpt_tokenizer(batch_texts, return_tensors="pt", padding=True, truncation=True, max_length=512).to(device)
            outputs = gpt_model(**inputs)

            logits = outputs.logits
            shift_logits = logits[..., :-1, :].contiguous()
            shift_labels = inputs["input_ids"][..., 1:].contiguous()
            shift_mask = inputs["attention_mask"][..., 1:].contiguous()
            log_probs = torch.log_softmax(shift_logits, dim=-1)

            for j in range(len(batch_texts)):
                valid_len = torch.sum(shift_mask[j]).item()
                if valid_len <= 0:
                    results.append(0)
                    continue
                token_log_probs = []
                for k in range(valid_len):
                    token_id = shift_labels[j, k].item()
                    lp = log_probs[j, k, token_id].item()
                    token_log_probs.append(-lp)
                avg_log_rank = sum(token_log_probs) / len(token_log_probs) if token_log_probs else 0
                results.append(avg_log_rank)

            del outputs, logits, log_probs, inputs
    return results

df['feat_81_logrank'] = calculate_logrank(texts, batch_size=8)

# FREE MEMORY
del gpt_model
del gpt_tokenizer
torch.cuda.empty_cache()
gc.collect()
print("✓ AraGPT2 features done & memory cleared.")


# ------------------------------------------------------------
# PART C: Stanza Features (Optimized Stream Mode)
# ------------------------------------------------------------
print("\n--- Running Stanza (POS Tagging) - STREAM MODE ---")

try:
    import stanza
    stanza.download('ar', verbose=False)
    nlp_stanza = stanza.Pipeline(lang='ar', processors='tokenize,pos', verbose=False, use_gpu=True)

    # Replace empty text with "." to keep alignment
    texts_for_stanza = [t if isinstance(t, str) and t.strip() else "." for t in df['text']]
    print(f"Streaming {len(texts_for_stanza)} texts to Stanza...")

    results = []
    for doc in tqdm(nlp_stanza.stream(texts_for_stanza), total=len(texts_for_stanza), desc="Feature 56"):
        adj = 0
        adv = 0
        for sent in doc.sentences:
            for word in sent.words:
                if word.upos == 'ADJ': adj += 1
                elif word.upos == 'ADV': adv += 1
        # Laplace Smoothing
        ratio = (adj + 1) / (adv + 1)
        results.append(ratio)
```

```
    df['feat_56_adj_adv_ratio'] = results
    print("✓ Stanza features done.")

except Exception as e:
    print(f"Stanza failed: {e}")
    df['feat_56_adj_adv_ratio'] = 0
```

```
Running on device: cuda

--- Loading AraBERT (for Features 59, 60 & 98) ---
tokenizer_config.json: 100%  381/381 [00:00<00:00, 40.8kB/s]
config.json: 100%  384/384 [00:00<00:00, 25.8kB/s]
vocab.txt:  825k/? [00:00<00:00, 24.4MB/s]
tokenizer.json:  2.64M/? [00:00<00:00, 84.9MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 7.98kB/s]
model.safetensors: 100%  543M/543M [00:03<00:00, 254MB/s]
✓ AraBERT Loaded (Float16).
Using RAW text for Deep Learning features...
Feature 59 & 60 (Embeddings): 100%|██████████| 41/41 [00:59<00:00,  1.45s/it]
Feature 98 (CLS Score): 100%|██████████| 82/82 [03:00<00:00,  2.21s/it]
✓ AraBERT features done & memory cleared.

--- Loading AraGPT2 (for Feature 81 LogRank) ---
config.json: 100%  843/843 [00:00<00:00, 87.8kB/s]
vocab.json:  1.94M/? [00:00<00:00, 42.9MB/s]
merges.txt:  1.50M/? [00:00<00:00, 68.9MB/s]
tokenizer.json:  4.52M/? [00:00<00:00, 104MB/s]
model.safetensors: 100%  1.50G/1.50G [00:16<00:00, 131MB/s]
✓ AraGPT2 Loaded (Float16).
Feature 81 (LogRank): 100%|██████████| 5243/5243 [15:29<00:00,  5.64it/s]
✓ AraGPT2 features done & memory cleared.

--- Running Stanza (POS Tagging) - STREAM MODE ---
Streaming 41937 texts to Stanza...
Feature 56: 100%|██████████| 41937/41937 [1:05:10<00:00, 10.72it/s]✓ Stanza features done.
```

```
# ==========================================
# PHASE 3: RESTORING MISSING FEATURES (14, 77, 102)
# ==========================================
print("\n--- Phase 3: Calculating Missing Statistical Features ---")

# 1. Feature 14: Hapax Dislegomena
def get_hapax_dislegomena(text):
    if not isinstance(text, str) or not text.strip():
        return 0.0
    words = text.split()
    counts = Counter(words)
    twice_count = sum(1 for count in counts.values() if count == 2)
    total_words = len(words)
    return twice_count / total_words if total_words > 0 else 0.0

print("Calculating Feature 14...")
df['feat_14_hapax_dislegomena'] = df['processed_text'].apply(get_hapax_dislegomena)


# 2. Feature 102: Detailed Diacritic Type Counts
DIACRITICS = {
    "fatha": "َ",
    "damma": "ُ",
    "kasra": "ِ",
    "sukun": "ْ",
    "shadda": "ّ",
    "tanwin_fath": "ً",
    "tanwin_damm": "ٌ",
    "tanwin_kasr": "ٍ"
}

print("Calculating Feature 102 (Detailed Diacritics)...")
```

```
# FIX: Use DIACRITICS dictionary, not the undefined 'diacritic_map'
for name, char in DIACRITICS.items():
    df[f'feat_102_{name}'] = df['text'].astype(str).apply(lambda x: x.count(char))


# 3. Feature 77: Corpus-Level Top 100 Words
print("Calculating Feature 77 (Global Top 100)...")

# Combine all processed text
all_text_combined = ' '.join(df['processed_text'].astype(str).tolist())
all_words = all_text_combined.split()
total_corpus_words = len(all_words)
word_counts_global = Counter(all_words)

# Get top 100 words
most_common_100 = word_counts_global.most_common(100)
top_100_freq_sum = sum(count for word, count in most_common_100)

# 1. Global Ratio
feature77_global_value = top_100_freq_sum / total_corpus_words if total_corpus_words > 0 else 0.0
df['feat_77_global_top100_ratio'] = feature77_global_value

# 2. Top 100 Words String (Comma separated list)
# ADDED: This extracts the actual words so they aren't lost, same as original code
top_100_words_list = [word for word, count in most_common_100]
top_100_str = ", ".join(top_100_words_list)
df['feat_77_top100_string'] = top_100_str # Assigned to the whole column (will repeat)

print("✓ Phase 3 Complete.")


# ------------------------------------------------------------
# FINAL SAVE
# ------------------------------------------------------------
print("\nSaving final dataset...")
df.to_excel("data/processed/Final_Engineered_Dataset.xlsx", index=False)
print("SUCCESS! File saved to: data/processed/Final_Engineered_Dataset.xlsx")
```

```
--- Phase 3: Calculating Missing Statistical Features ---
Calculating Feature 14...
Calculating Feature 102 (Detailed Diacritics)...
Calculating Feature 77 (Global Top 100)...
✓ Phase 3 Complete.

Saving final dataset...
SUCCESS! File saved to: data/processed/Final_Engineered_Dataset.xlsx
```

```
from IPython.display import display
```

```
# ==========================================
# TASK 2.2: SUPERIOR EDA & REPORT ZIPPING (REFACTORED & FIXED)
# ==========================================

# 1. SETUP & LIBRARIES
print("1. Installing visualization libraries...")
!pip install -q arabic-reshaper python-bidi wordcloud seaborn

# Download a proper Arabic font (Amiri) to avoid empty boxes in WordCloud
!wget -q -O amiri_font.ttf https://github.com/google/fonts/raw/main/ofl/amiri/Amiri-Regular.ttf

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import seaborn as sns
import re
import os
import zipfile
import warnings
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from wordcloud import WordCloud
import arabic_reshaper
from bidi.algorithm import get_display
```

```
from nltk.corpus import stopwords
import nltk

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

os.makedirs('reports/figures', exist_ok=True)
nltk.download('stopwords', quiet=True)

# Register Arabic font for Matplotlib
fm.fontManager.addfont('amiri_font.ttf')
plt.rcParams['font.family'] = 'Amiri'
plt.rcParams['figure.dpi'] = 120
sns.set_style("whitegrid")

# Helper for Arabic reshaping
def reshaper(text):
    if not isinstance(text, str): return str(text)
    reshaped_text = arabic_reshaper.reshape(text)
    return get_display(reshaped_text)

# 2. LOAD DATA (User Provided Logic)
file_path = "/content/Final_Engineered_Dataset.xlsx"
print(f"\n2. Loading data from {file_path}...")

if os.path.exists(file_path):
    df_eda = pd.read_excel(file_path)
else:
    # Try CSV fallback
    csv_path = "data/processed/Final_Engineered_Dataset.csv" # Fixed typo from .xslx to .csv
    if os.path.exists(csv_path):
        df_eda = pd.read_csv(csv_path)
    else:
        raise FileNotFoundError(f"CRITICAL: Data file not found at {file_path} or {csv_path}")

print(f"Loaded {len(df_eda)} rows.")

# ============================================================
# PART A: RAW DATA HEALTH CHECK & STRANGE CHARACTERS
# ============================================================
print("\n--- A. Raw Data Health Check ---")

# 1. Strange Character Extraction
def extract_strange_list(text):
    if pd.isna(text): return []
    # Regex: Anything NOT Arabic, NOT Number, NOT Space
    return re.findall(r"[^0-9\u0600-\u06FF\s]", str(text))

print("Extracting strange symbols from raw text...")
all_strange_tokens = [token for text in df_eda['text'] for token in extract_strange_list(text)]

# Count frequencies
strange_counts = Counter(all_strange_tokens)
total_strange_count = sum(strange_counts.values())
unique_strange_count = len(strange_counts)

print(f"Found {unique_strange_count} unique strange characters. Total occurrences: {total_strange_count}")

# 2. Save Strange Characters to CSV
df_strange = pd.DataFrame(strange_counts.most_common(), columns=['Character', 'Count'])
str_csv_path = 'reports/strange_characters.csv'
df_strange.to_csv(str_csv_path, index=False)
print(f"Saved strange characters to '{str_csv_path}'")

# 3. General Stats CSV
print("Generating General Info Report...")
health_data = {
    'Metric': ['Total Rows', 'Human Samples', 'AI Samples', 'Missing Values', 'Duplicates', 'Total Strange Chars'],
    'Value': [
        len(df_eda),
        df_eda[df_eda['label']=='human'].shape[0],
        df_eda[df_eda['label']=='ai'].shape[0],
        df_eda['text'].isnull().sum(),
        df_eda.duplicated(subset=['text']).sum(),
        total_strange_count
    ]
}
pd.DataFrame(health_data).to_csv('reports/raw_data_health_check.csv', index=False)
```

```
pd.DataFrame(health_data).to_csv('reports/raw_data_health_check.csv', index=False)

# ============================================================
# PART B: VISUALIZATION (STRANGE CHARACTERS)
# ============================================================
print("\n--- B. Visualizing Strange Characters ---")

def plot_top_strange(data, top_n, filename, height):
    if data.empty: return

    subset = data.head(top_n)

    plt.figure(figsize=(10, height))
    # Fix: Added hue=Character and legend=False to silence FutureWarning
    sns.barplot(data=subset, y='Character', x='Count', palette='viridis', hue='Character', legend=False)
    plt.title(f"Top {top_n} Strange Characters/Symbols")
    plt.xlabel("Frequency")
    plt.ylabel("Character/Symbol")

    # Add labels
    for i, v in enumerate(subset['Count']):
        plt.text(v, i, f" {v}", va='center', fontsize=9)

    plt.tight_layout()
    plt.savefig(f'reports/figures/{filename}')
    plt.close()
    print(f"Saved: reports/figures/{filename}")

# Generate Graphs (Added Top 10)
plot_top_strange(df_strange, 10, 'strange_chars_top_10.png', 6)
plot_top_strange(df_strange, 25, 'strange_chars_top_25.png', 8)
plot_top_strange(df_strange, 50, 'strange_chars_top_50.png', 12)
plot_top_strange(df_strange, 100, 'strange_chars_top_100.png', 20)

# ============================================================
# PART C: ADVANCED LINGUISTIC ANALYSIS
# ============================================================
print("\n--- C. Advanced Linguistic Analysis ---")

def calc_text_stats(text):
    if not isinstance(text, str): return pd.Series([0, 0, 0])
    words = text.split()
    if len(words) == 0: return pd.Series([0, 0, 0])
    avg_word_len = sum(len(w) for w in words) / len(words)
    ttr = len(set(words)) / len(words)
    sentences = re.split(r'[.!?؟]', text)
    avg_sent_len = len(words) / len(sentences) if len(sentences) > 0 else len(words)
    return pd.Series([avg_word_len, ttr, avg_sent_len])

df_eda[['avg_word_len', 'ttr', 'avg_sent_len']] = df_eda['processed_text'].apply(calc_text_stats)

stats_summary = df_eda.groupby('label')[['avg_word_len', 'ttr', 'avg_sent_len']].mean()
stats_summary.to_csv('reports/linguistic_stats_summary.csv')
display(stats_summary)

# ------------------------------------------------------------
# PART D: PUNCTUATION (Log Scale)
# ------------------------------------------------------------
print("\n--- D. Punctuation Analysis ---")
def count_punct(text):
    return len(re.findall(r'[.,،؟!:؛?]', str(text)))

df_eda['punct_count'] = df_eda['text'].apply(count_punct)
df_eda['punct_density'] = (df_eda['punct_count'] / df_eda['text'].str.len()) * 1000

plt.figure(figsize=(8, 5))
# Fix: Added hue=label
sns.boxplot(data=df_eda, x='label', y='punct_density', palette='Pastel1', hue='label', legend=False)
plt.yscale('symlog')
plt.title("Punctuation Density (Log Scale)")
plt.savefig('reports/figures/eda_punctuation_log_scale.png')
plt.close()

# ------------------------------------------------------------
# PART E: HEATMAP
# ------------------------------------------------------------
print("\n--- E. Feature Correlations ---")
label_map = {'ai': 0, 'human': 1}
```

```
df_eda['label_encoded'] = df_eda['label'].map(label_map)

feat_cols = [c for c in df_eda.columns if c.startswith('feat_') and pd.api.types.is_numeric_dtype(df_eda[c])]
cols_to_corr = ['label_encoded'] + feat_cols

if len(feat_cols) > 0:
    plt.figure(figsize=(12, 10))
    corr_matrix = df_eda[cols_to_corr].corr()
    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
    sns.heatmap(corr_matrix, mask=mask, annot=False, cmap='coolwarm', linewidths=0.5, center=0)
    plt.title("Feature Correlation Matrix (AI=0, Human=1)")
    plt.savefig('reports/figures/eda_feature_correlation_matrix.png')
    plt.close()

# ------------------------------------------------------------
# PART F: PCA PROJECTION
# ------------------------------------------------------------
print("\n--- F. PCA Projection ---")
if len(feat_cols) > 2:
    X = df_eda[feat_cols].fillna(0)
    y = df_eda['label']
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled)

    plt.figure(figsize=(10, 7))
    sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, alpha=0.6, palette='viridis', s=15)
    plt.title(f"PCA Projection\nExplained Variance: {sum(pca.explained_variance_ratio_):.2%}")
    plt.savefig('reports/figures/eda_pca_projection.png')
    plt.close()

# ------------------------------------------------------------
# PART G: DISTINCTIVE WORDS
# ------------------------------------------------------------
print("\n--- G. Distinctive Vocabulary ---")
human_idx = df_eda['label'] == 'human'
ai_idx = df_eda['label'] == 'ai'

if human_idx.sum() > 0 and ai_idx.sum() > 0:
    tfidf = TfidfVectorizer(max_features=5000, stop_words=stopwords.words('arabic'))
    X_tfidf = tfidf.fit_transform(df_eda['processed_text'].astype(str))
    words = np.array(tfidf.get_feature_names_out())

    # Safe slicing
    diff = X_tfidf[human_idx.values].mean(axis=0).A1 - X_tfidf[ai_idx.values].mean(axis=0).A1

    top_human = diff.argsort()[-10:][::-1]
    top_ai = diff.argsort()[:10]

    fig, ax = plt.subplots(1, 2, figsize=(14, 6))
    h_words = [reshaper(words[i]) for i in top_human]
    a_words = [reshaper(words[i]) for i in top_ai]

    # Fix: Added hue for barplots
    sns.barplot(x=diff[top_human], y=h_words, ax=ax[0], palette="Greens_r", hue=h_words, legend=False)
    ax[0].set_title("Most Distinctive HUMAN Words")

    sns.barplot(x=np.abs(diff[top_ai]), y=a_words, ax=ax[1], palette="Blues_r", hue=a_words, legend=False)
    ax[1].set_title("Most Distinctive AI Words")
    plt.tight_layout()
    plt.savefig('reports/figures/eda_distinctive_vocab.png')
    plt.close()

# ------------------------------------------------------------
# PART H: TOP BIGRAMS
# ------------------------------------------------------------
print("\n--- H. Top Bigrams ---")
def plot_save_ngrams(text_series, label, color_pal):
    if len(text_series) == 0: return
    vec = CountVectorizer(ngram_range=(2, 2), max_features=10)
    try:
        bow = vec.fit_transform(text_series.astype(str))
        sum_words = bow.sum(axis=0)
        words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
        words_freq = sorted(words_freq, key = lambda x: x[1], reverse=True)
```

```
        words, freqs = zip(*words_freq)
        words_display = [reshaper(w) for w in words]

        plt.figure(figsize=(8, 5))
        # Fix: Added hue
        sns.barplot(x=list(freqs), y=list(words_display), palette=color_pal, hue=list(words_display), legend=False)
        plt.title(f"Top Bigrams: {label.upper()}")
        plt.tight_layout()
        plt.savefig(f'reports/figures/eda_bigrams_{label}.png')
        plt.close()
    except ValueError: pass

plot_save_ngrams(df_eda[df_eda['label']=='human']['processed_text'], 'human', 'Greens_r')
plot_save_ngrams(df_eda[df_eda['label']=='ai']['processed_text'], 'ai', 'Blues_r')

# ------------------------------------------------------------
# PART I: WORD CLOUDS
# ------------------------------------------------------------
print("\n--- I. Generating Word Clouds ---")
def generate_wc(text_series, filename, color_map, bg):
    full_text = " ".join(text_series.astype(str))
    counts = Counter(full_text.split())
    if not counts: return

    reshaped_counts = {reshaper(k): v for k, v in counts.items()}

    wc = WordCloud(font_path='amiri_font.ttf', width=800, height=400,
                   background_color=bg, colormap=color_map, regexp=r"[\u0600-\u06FF]+").generate_from_frequencies(reshaped_counts
    wc.to_file(filename)

    plt.figure(figsize=(10, 5))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    plt.title(f"Word Cloud: {filename.split('_')[-1].split('.')[0].upper()}")
    plt.close()

generate_wc(df_eda[df_eda['label']=='human']['processed_text'], 'reports/figures/eda_wordcloud_human.png', 'viridis', 'white')
generate_wc(df_eda[df_eda['label']=='ai']['processed_text'], 'reports/figures/eda_wordcloud_ai.png', 'Pastel1', 'black')

# ==========================================
# FINAL STEP: ZIP REPORTS
# ==========================================
print("\n" + "="*40)
print("   ZIPPING REPORTS ONLY")
print("="*40)

target_directory = 'reports'
output_zip = 'Step1_Reports_Only_PP_FE.zip'

with zipfile.ZipFile(output_zip, 'w', zipfile.ZIP_DEFLATED) as zipf:
    if os.path.exists(target_directory):
        for root, dirs, files in os.walk(target_directory):
            dirs[:] = [d for d in dirs if not d.startswith('.')]
            for file in files:
                if not file.startswith('.'):
                    file_path = os.path.join(root, file)
                    zipf.write(file_path, arcname=file_path)
                    print(f"  Added: {file_path}")

print(f"\nSUCCESS! Download '{output_zip}'")
try:
    from google.colab import files
    files.download(output_zip)
except: pass
```

```
1. Installing visualization libraries...

2. Loading data from /content/Final_Engineered_Dataset.xlsx...
Loaded 41937 rows.

--- A. Raw Data Health Check ---
Extracting strange symbols from raw text...
Found 368 unique strange characters. Total occurrences: 268918
Saved strange characters to 'reports/strange_characters.csv'
Generating General Info Report...

--- B. Visualizing Strange Characters ---
Saved: reports/figures/strange_chars_top_10.png
Saved: reports/figures/strange_chars_top_25.png
Saved: reports/figures/strange_chars_top_50.png
Saved: reports/figures/strange_chars_top_100.png

--- C. Advanced Linguistic Analysis ---
```

label	avg_word_len	ttr	avg_sent_len
ai	3.148299	0.688060	86.162151
human	3.190100	0.725091	96.915713

```
--- D. Punctuation Analysis ---

--- E. Feature Correlations ---

--- F. PCA Projection ---

--- G. Distinctive Vocabulary ---

--- H. Top Bigrams ---

--- I. Generating Word Clouds ---

========================================
   ZIPPING REPORTS ONLY
========================================
  Added: reports/strange_characters.csv
  Added: reports/linguistic_stats_summary.csv
  Added: reports/raw_data_health_check.csv
  Added: reports/figures/eda_distinctive_vocab.png
  Added: reports/figures/strange_chars_top_25.png
  Added: reports/figures/eda_bigrams_ai.png
  Added: reports/figures/strange_chars_top_10.png
  Added: reports/figures/eda_feature_correlation_matrix.png
  Added: reports/figures/eda_wordcloud_human.png
  Added: reports/figures/eda_punctuation_log_scale.png
  Added: reports/figures/eda_pca_projection.png
  Added: reports/figures/eda_wordcloud_ai.png
  Added: reports/figures/strange_chars_top_50.png
  Added: reports/figures/strange_chars_top_100.png
  Added: reports/figures/eda_bigrams_human.png

SUCCESS! Download 'Step1_Reports_Only_PP_FE.zip'
```

Next steps: Generate code with `stats_summary` | New interactive sheet